

Campus Marketplace

Project Prototype Report for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Team Enigma*

September 7, 2026

1 Time-to-value

- We delivered meaningful increments quickly by prototyping the core user workflow and validating it with users within a short iteration window.
- We prioritized fast feedback over perfection by using weekly iterations to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on user feedback.

2 Problem Statement

Our campus (and college campuses in general) generates a constant churn of physical items, students arrive with belongings, need more as the semester goes on (especially during the first semester), and leave most of it behind or discard it when they graduate. This creates two connected but distinct problems:-

- **Inefficient buying and selling of used items:** Students frequently need affordable second-hand items (like bicycles, furniture, textbooks, appliances), but have no centralized, trustworthy platform to find them within this campus community. The current options we have are general marketplaces like Facebook Marketplace or OLX. These are not campus-specific and involve dealing with strangers off-campus, lack verification of buyers/sellers, and make logistics (meeting, payment, delivery) needlessly complicated for what should be a short walk across campus. Alternatively, students rely on social media posts or simply asking around, but these methods have limited reach and rarely produce enough options to compare. So, it makes it hard to find the best variety or the most affordable deal.

*Aditya Kaushik, Roll No: 1024030025, Email: aaditya_be24@thapar.edu,
Priyansh Singh Chouhan, Roll No: 1024030929, Email: pchouhan_be24@thapar.edu

- **No easy way to borrow rarely-used items:** Many items students own are needed only occasionally, like a drill, a gaming controller, a formal blazer for one event, project-specific equipment, etc. Buying these outright is wasteful and expensive, yet there is no structured system for students to lend and borrow such items within this campus network. As a result, students either overspend on things they will barely use or have to source it manually through word-of-mouth or social media apps.

3 Implemented Solution

3.1 Overview

We built Campus Marketplace, a campus-exclusive platform restricted to @thapar.edu accounts that unifies two modes of resource exchange in one system: permanent buying/selling of used items and temporary borrowing/lending of items needed only occasionally. Every listing is created as either a SELL or BORROW type, each with its own pricing model (price for sale, price per day for borrowing), so the same underlying platform serves both use cases without becoming two disconnected apps.

3.2 Core Workflow

1. **Verified onboarding:** Users sign up with their Thapar email (@thapar.edu only), linked to a unique user profile before they can list, request, or borrow anything. Figure 2 shows the implemented authentication screen.
2. **Listing creation:** A user posts an item under a category, with a title, description, type (SELL/BORROW), price or daily rate, and photos. Once created, a listing is immutable, meaning sellers relist rather than edit their posts, keeping transaction history accurate and disputes traceable. Figure 3 shows the listing creation form.
3. **Buying flow:** Interested buyers raise a purchase request on a listing. The owner can approve or reject it. If approved, it auto-generates a transaction and auto-rejects any other pending requests on that listing, since an item can only be sold once.
4. **Borrowing flow:** Interested borrowers propose a start and end time on a BORROW listing. Once booked, a borrowing record is created with an auto-computed total amount ($price_per_day \times days$), and the system blocks any overlapping booking requests for that same listing.
5. **Hostel-based discovery:** Listings are intended to be filterable by hostels, so users can search and filter for items in their own vicinity, making handoffs quick and easy.
6. **Due-date reminders:** For active borrowings, the website is intended to send automated reminders as the end time approaches, and a follow-up nudge if an item isn't marked returned on time, reducing the need for lenders to chase borrowers manually.
7. **Closure and trust-building:** Once a transaction is completed or a borrowing is returned, both parties are intended to be able to leave a rating and review of each other, building a visible trust score over time.

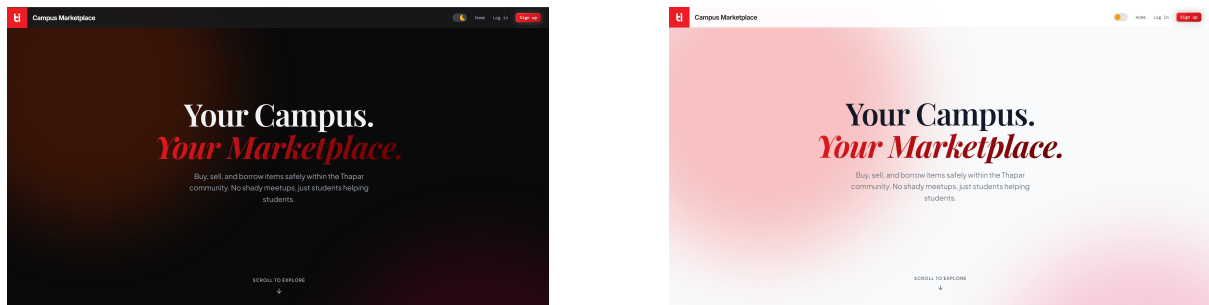


Figure 1: Homepage in dark mode (left) and light mode (right), demonstrating the fully implemented theme toggle

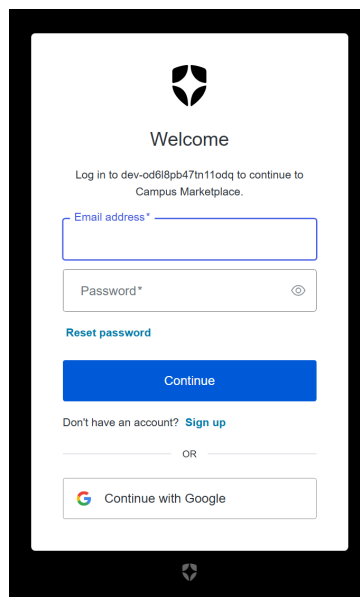


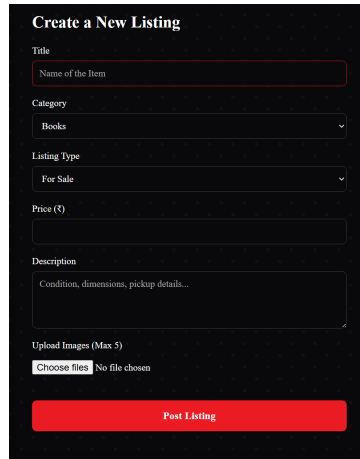
Figure 2: Verified onboarding: campus email authentication screen

3.3 Implementation Status

Table 1 summarizes the implementation status of each core workflow feature at the time of this report.

Feature	Status
Verified onboarding	Implemented
Listing creation	Implemented
Buying flow	Implemented
Borrowing flow	Implemented
Hostel-based discovery	Planned
Due-date reminders	Planned
Closure and trust-building (reviews)	Planned

Table 1: Implementation status of core workflow features



Create a New Listing

Title
Name of the Item

Category
Books

Listing Type
For Sale

Price (₹)
₹

Description
Condition, dimensions, pickup details...

Upload Images (Max 5)
Choose files No file chosen

Post Listing

Figure 3: Listing creation form with category, type, price, and image upload

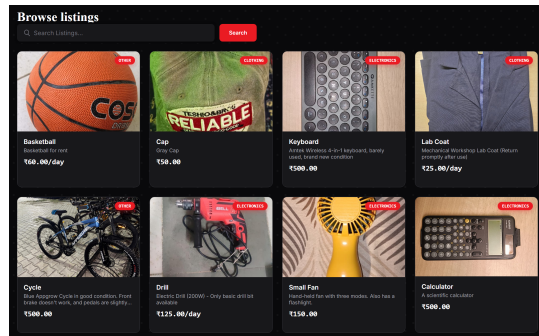


Figure 4: Browse listings page with search bar

Implementation Notes

Development effort was prioritized around the core transaction pipeline first: verified onboarding, listing creation, and the buying/borrowing request-and-approval flows, since these define the platform's fundamental data integrity guarantees (immutable listings, non-overlapping bookings, single-transaction-per-listing constraints). Hostel-based discovery, due-date reminders, and the review/trust-scoring system are deferred to the next development milestone. These were sequenced after the core flows because they depend on stable transaction and borrowing records already existing in the system (e.g., reviews require completed transactions to reference, and reminders require finalized borrowing end-times to schedule against). Additionally, the platform supports a fully functional light/dark theme toggle (Figure 1), allowing users to switch appearance preferences across the site.

Figure 5: Listing filters: category, listing type (Sell/Borrow), and price range

3.4 Operational Constraints

To keep the system consistent and abuse-resistant, a few rules are enforced at the data/business-logic level:

- A listing can carry either a sale price or a daily rate, never both.
- Status transitions (for a borrowing or purchase request) follow a fixed state machine. An item can't jump from pending state straight to returned/completed state, skipping the intermediate states.
- Each listing can only ever produce one final transaction. The transaction amount must match the listing's price, and the recorded seller must be the item's actual owner.
- A buyer/listing pair can have at most one pending and one accepted request at a time.
- Reviews are only permitted once a deal is closed (completed transaction or returned borrowing), and only from the two actual parties involved, i.e. one review per person per deal (*not yet implemented*).

4 Solution Approach

This is an engineering course project, so we focused on scalable administrative and workflow aspects rather than novel algorithm research.

- **Identity-gated access as the trust layer:** Instead of building verification systems from scratch, campus email verification is the single source of trust. It is cheap to enforce, hard to fake, and immediately scoped to the right user base.

- **State-machine-driven transactions:** Purchases and borrowings move through explicit, restricted status transitions rather than free-form updates. This makes the system's behavior predictable, auditable, and easy to extend without rewriting core logic.
- **Immutable listings for accountability:** Treating listings as append-only (relist instead of edit) keeps a clean historical record. This is useful for resolving disputes, generating trust scores, and future admin auditing, at negligible engineering cost.
- **Conflict prevention by design:** Overlapping booking checks and single transaction per listing rules prevent double-selling or double-booking at the data layer itself, rather than requiring manual moderation to catch these cases after the fact.
- **Location-aware filtering (*planned*):** Hostel filtering is designed as a simple, scalable filter on existing listing data, improving discovery without needing any recommendation engine.
- **Reminders as scheduled jobs (*planned*):** Return-date nudges are designed to run as simple scheduled checks against end time values, keeping the borrowing flow self-managing without needing constant lender follow-up or admin intervention.
- **Thin backend, managed identity and storage:** Node.js handles routes, business logic, and validation, while Auth0 and Supabase offload authentication and file storage respectively, reducing the surface area the team needs to build and maintain themselves, so effort stays focused on marketplace logic rather than infrastructure.

5 Tech Stack

The following stack was used to implement the platform. Note that the authentication and storage choices deviated from the original plan, as discussed below.

- **Frontend (React + Vite):** A static, decoupled layer served independently of backend uptime, so cached pages can degrade gracefully rather than fail completely during backend maintenance. Vite was used as the build tool for faster development and build times.
- **Backend (Node.js):** Stateless by design (routes, controllers, business logic, validation). No session state lives on the server itself, so it can be horizontally scaled or restarted without losing in-flight user context.
- **Auth0:** Used for authentication instead of Supabase Auth, as originally planned. Auth0 was chosen for its greater scalability and stricter enforcement of authentication policies, which better suited the campus-email-restricted signup flow.
- **Supabase:** Used for storing listing images, rather than for authentication as originally proposed. This keeps file storage on a managed service while authentication is handled by a dedicated, more specialized provider.
- **PostgreSQL:** Used for core data storage. Since listings are immutable and status transitions are restricted, the data layer stays consistent even under concurrent access, which matters more for reliability than raw uptime percentage at this scale.

Implementation Notes

The original proposal called for Supabase to handle both authentication and file storage. During implementation, authentication was moved to Auth0 to take advantage of its more mature scalability guarantees and stricter policy enforcement, both of which were considered important given the platform's reliance on verified campus-email identity as its sole trust layer. Supabase was retained for its original strength, i.e. managed file storage, and is now scoped specifically to listing images.

6 Future Scalability Considerations

The current implementation targets one campus (Thapar) as an isolated community. The following refinements are worth considering for extending the platform to multiple campuses in the future:

- **Data isolation via campus scoping, not separate deployments:** Rather than literally standing up a new website per college (which multiplies infrastructure and maintenance cost), the schema can add a campus id to users and scope every query by it. One shared backend and database would serve all campuses, but a Thapar user would only ever see Thapar listings. This would keep the same operational cost as one website, with the same trust boundary as separate ones.
- **Email-domain-based campus binding:** Since trust is already anchored to verified college email (@thapar.edu), each new campus could simply register its email domain, with isolation enforced at signup. This would remove the need for separate infrastructure per campus.
- **Horizontal scaling of the backend:** Since Express handlers are stateless, adding campuses would mainly add read/write load, which could be handled by scaling backend instances behind a load balancer, rather than needing a redesign.
- **Shared category layer:** Categories could stay shared, so the team would not need to maintain duplicate reference data per campus.
- **Growth path:** The project began with one campus (Thapar) to validate the workflow; additional campuses could be onboarded as isolated logical communities within the same system. This would avoid re-architecting later and would keep a single codebase, single admin panel, and single set of integrity rules enforced everywhere.

7 Evaluation Criterion

Success can be measured across a few dimensions. Note that some metrics depend on features currently marked as planned (Table 1) and will only be measurable once those are implemented.

- **Adoption:** Number of verified users per campus, active listings per week, ratio of listings created to listings actually resulting in a transaction/borrowing.
- **Trust and reliability** (*depends on reviews, planned*): Average rating/review scores, dispute rate (e.g. items not returned, mismatched transaction amounts), percentage of borrowings returned on time versus requiring reminder escalation.

- **Efficiency of exchange:** Average time from listing creation to the request being accepted, and time from request to transaction completion.
- **Resource utilization:** Ratio of borrow transactions to sell transactions (a good split reflects the platform genuinely encouraging sharing over unnecessary buying, which supports sustainability).
- **System health** (*job success rate depends on reminders, planned*): Request latency, job success rate for reminders, and constraint-violation rate (how often the state-machine/integrity rules catch invalid transitions).

8 Higher-Order Goal

Beyond the immediate utility of “buy, sell, borrow items on campus,” the platform’s broader aim is to normalize resource-sharing as the default behavior within a trusted community, rather than an occasional favor asked of friends. By reducing the friction, risk, and search cost involved in buying second-hand or borrowing instead of purchasing new, the website aims to:

- Cut down unnecessary spending and waste generated by short-lived student needs.
- Extend the useful life of physical items by keeping them circulating within a community instead of ending up discarded when a student graduates.
- Improve trust, accountability and sustainability of the community.

9 Project Scope and Deliverables

9.1 Completed

- Core DB schema with integrity rules enforced.
- Student login authentication (via Auth0) and profile setup.
- Create, browse, and search listings.
- Transaction constraints and implementation based on type of request (Buy or Borrow).

Screenshots of the completed onboarding and listing creation flows are provided in Section 3 (Figures 2 and 3).

9.2 Not Completed / Deferred

- Hostel-based filtering.
- Automated return-date reminders.
- Ratings and reviews post transaction/return.

The deferred items were sequenced after the core transaction pipeline, since they depend on stable transaction/borrowing records already existing in the system: reviews require a completed transaction or return to reference, and reminders require finalized borrowing end-times to schedule against. Hostel-based filtering was deprioritized in favor of hardening the core buy/borrow logic first. These items are planned for the next development phase.

10 Risks Encountered and Actual Mitigation

- **Low initial adoption of the website:** Mitigated by seeding launch in our own hostel and encouraging early listings before public rollout.
- **Items not returned on time or borrower disputes:** Automated reminders and review-based flagging of repeat offenders were planned mitigations for this risk, but both remain deferred (Section 9). In their absence, borrowing status is currently tracked manually against end-time records, which partially addresses the risk but does not fully replace automated escalation.
- **Fake or duplicate accounts:** Mitigated by restricting signup to verified @thapar.edu email addresses, enforced via Auth0.
- **Data inconsistency:** Mitigated by enforcing integrity rules such as overlap checks and restricted status transitions directly at the database layer.
- **Users bypassing the website after first contact:** Mitigated by keeping the website flow low-friction enough to remain the easiest path, rather than something users work around.

11 Summary

Campus Marketplace is a campus-exclusive marketplace that unifies buying-selling and borrowing-lending of everyday items within a verified student community. Built on a React (Vite) and Node.js stack, with Auth0 for authentication and Supabase for listing image storage, the platform enforces trust and data integrity through email verification, immutable listings, and restricted status transitions rather than complex algorithms. The core transaction pipeline, covering onboarding, listing creation, and buy/borrow request flows, has been implemented and tested, while hostel-based filtering, automated return reminders, and post-transaction reviews remain deferred to a future development phase. Even in its current form, the platform reduces friction in peer-to-peer exchange within the Thapar community, with a scoped design that can extend to multiple campuses or similar trusted communities over time.

12 Results and Testing

12.1 Implemented Results

The implemented prototype successfully supports the primary marketplace workflow from user authentication to initiating a purchase or borrowing request. Users can authenticate using their institutional email, create listings, browse available listings, and interact with listings through the appropriate buying or borrowing flow.

The implemented system currently provides the following functionality:

- *Institutional authentication:* Users are restricted to verified @thapar.edu accounts through Auth0.
- *User profile setup:* Each authenticated user is associated with a unique profile.
- *Listing creation:* Users can create SELL or BORROW listings with a title, description, category, pricing information, and images.

- *Listing discovery*: Users can browse and search available listings.
- *Purchase requests*: Buyers can request an item, after which the owner can approve or reject the request.
- *Borrowing requests*: Users can specify borrowing start and end times for BORROW listings.
- *Transaction generation*: An accepted purchase request generates a corresponding transaction.
- *Borrowing amount calculation*: The total borrowing amount is calculated from the daily rate and borrowing duration.
- *Data integrity constraints*: The system prevents invalid state transitions, conflicting borrowing periods, and multiple final transactions for the same listing.
- *Theme support*: The interface provides both light and dark modes, as demonstrated by the homepage screenshots in Figure 1.

The completed implementation therefore covers the fundamental exchange pipeline, while hostel-based filtering, automated reminders, and post-transaction reviews remain outside the current prototype scope.

12.2 Testing Performed

Testing focused primarily on the core transaction and data-integrity rules rather than performance benchmarking. Table 2 summarizes the test cases used to verify expected system behaviour.

Test Case	Expected Result	Status
Sign up using valid @thapar.edu email	User is successfully authenticated	Passed
Attempt signup using non-campus email	Authentication is rejected	Passed
Create a SELL listing	Listing is created with sale price	Passed
Create a BORROW listing	Listing is created with daily borrowing rate	Passed
Submit a purchase request	Request is created in pending state	Passed
Owner accepts purchase request	Transaction is generated	Passed
Owner rejects purchase request	Request is rejected without creating transaction	Passed
Multiple purchase requests for one item	Once one is accepted, other pending requests are rejected	Passed
Submit overlapping borrowing request	Conflicting booking is rejected	Passed
Submit non-overlapping borrowing request	Request can proceed normally	Passed
Attempt invalid status transition	Operation is rejected	Passed
Attempt to assign both sale price and borrowing rate	Invalid listing is rejected	Passed
Browse/search existing listings	Matching listings are displayed	Passed
Toggle light/dark mode	Interface switches theme	Passed

Table 2: Test cases and results

These tests correspond to the integrity rules described in the implementation, particularly the single-transaction-per-listing constraint, restricted state transitions, and overlap prevention for borrowing requests.

12.3 Sample Data

For functional testing, the system was populated with representative campus-use items such as textbooks, bicycles, furniture, appliances, and other student-use resources. Both SELL and BORROW listing types were tested to ensure that the platform correctly handles their different pricing and transaction flows.

The screenshots in Section 3 demonstrate the resulting interface, including the authentication screen, listing creation form, browse listings page, and listing filters.

13 Challenges Faced and Design Decisions

13.1 Authentication Architecture Change

One of the major design changes during implementation was the decision to move authentication from Supabase Auth to Auth0. The original architecture planned to use Supabase for both authentication and file storage. During implementation, the team decided that Auth0 was better suited to the project's requirement for institutional-email-based authentication and stricter authentication policy enforcement. Supabase was therefore retained specifically for listing-image storage.

This change required separating authentication responsibilities from storage responsibilities in the overall architecture, but resulted in a cleaner division between identity management and application data.

13.2 Preventing Double Booking

Borrowing introduced a more complicated constraint than ordinary buying because the same item can be requested multiple times for different time periods. A simple “available/unavailable” flag would not be sufficient because an item could be available for one period while already reserved for another.

The system therefore uses start and end times to detect overlapping borrowing requests. This prevents two accepted borrowing periods from occupying the same time interval. The overlap-prevention rule was treated as a data-integrity requirement rather than something left entirely to the user interface.

13.3 Maintaining Transaction Consistency

Another challenge was ensuring that transactions could not reach logically invalid states. Rather than allowing arbitrary status updates, purchase and borrowing requests follow predefined state transitions.

For example, a request cannot directly move from a pending state to a completed or returned state without passing through the appropriate intermediate state. Similarly, a listing can produce only one final transaction. These restrictions reduce the possibility of inconsistent records when multiple users interact with the same listing.

13.4 Immutable Listings

The team also chose not to allow users to modify listings after creation. Initially, allowing edits may appear simpler, but it introduces ambiguity if the price, item description, or ownership information changes after requests have already been created.

The implemented approach treats listings as effectively immutable: if an owner needs to change a listing, the intended approach is to create a new listing instead. This preserves a clearer historical record and simplifies future auditing and dispute resolution.

13.5 Feature Prioritization

Not every proposed feature could be implemented within the prototype milestone. Hostel-based discovery, automated return reminders, and ratings/reviews were deliberately deferred rather than implemented as incomplete versions.

The team prioritized the core transaction pipeline first because the deferred features depend on stable transaction and borrowing records. For example, reviews require a completed transaction or returned borrowing, while reminders require reliable borrowing end times.

This allowed the prototype to establish the fundamental data and workflow architecture before adding secondary functionality.

14 Individual and Team Contribution

Team Member	Contribution
Aditya Kaushik	Backend development, database schema and integrity constraints, transaction and borrowing logic, authentication integration, testing and debugging
Priyansh Singh Chouhan	Frontend development, UI/UX implementation, listing and browsing interfaces, integration testing and documentation

Table 3: Individual contributions

Both team members contributed to requirements analysis, architectural decisions, debugging, testing, and preparation of the final project report. Major design decisions, including the separation of authentication and storage responsibilities and prioritization of the core transaction pipeline, were discussed and evaluated jointly.

15 Conclusion and Future Work

15.1 Conclusion

Campus Marketplace demonstrates a working prototype for facilitating trusted peer-to-peer exchange within a university environment. The system combines buying, selling, borrowing, and lending into a single campus-restricted platform rather than treating these as separate applications.

The primary objective of the implementation was to establish a reliable transaction pipeline. This was achieved through institutional authentication, structured listings, controlled request states, immutable listing records, borrowing-overlap prevention, and database-level integrity constraints. The resulting architecture focuses on predictable system behaviour and data consistency rather than unnecessary algorithmic complexity.

The project also demonstrated the importance of prioritizing core functionality during prototype development. Instead of implementing every proposed feature superficially, the team completed the fundamental buy/borrow workflow and deferred features that depend upon it.

15.2 Future Work

The next development phase would focus on completing the deferred functionality:

1. **Hostel-based filtering** — allowing users to prioritize listings located near their hostel.
2. **Automated return reminders** — sending notifications as borrowing deadlines approach and escalating overdue returns.

3. **Ratings and reviews** — allowing completed transactions and returned borrowings to contribute to user trust scores.
4. **Improved dispute handling** — introducing mechanisms for reporting problematic transactions or repeated non-return behaviour.
5. **Administrative controls** — providing administrators with tools for moderation and handling reported listings/users.
6. **Multi-campus support** — introducing a campus identifier into the data model so multiple colleges can operate as logically isolated communities on the same infrastructure.
7. **Deployment and scalability** — deploying the frontend, backend, database, and storage services in a production environment and evaluating performance under a larger number of concurrent users.

The existing architecture is intended to support this expansion without requiring separate deployments for every campus. Campus-level data isolation and email-domain-based onboarding can instead be incorporated into the existing shared architecture.

16 References

1. *React Documentation* — React framework documentation and component-based UI development.
2. *Vite Documentation* — Build tooling and development environment used for the frontend.
3. *Node.js Documentation* — Runtime environment used for backend development.
4. *Auth0 Documentation* — Authentication and identity-management implementation.
5. *Supabase Documentation* — Managed storage used for listing images.
6. *PostgreSQL Documentation* — Relational database and data-integrity functionality.